

Robustness of Learning-Based Vulnerability Detectors under Semantics-Preserving Code Transformations

Phat T. Tran-Truong^{a,b}, Xuan-Bach Le^{a,b}

^aFaculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), 268 Ly Thuong Kiet Street, Dien Hong Ward, Ho Chi Minh City, Vietnam

^bVietnam National University Ho Chi Minh City, Linh Xuan Ward, Ho Chi Minh City, Vietnam

ARTICLE INFO

Keywords:

software vulnerability detection
robustness
empirical software engineering
software quality assurance
code transformation
software security
metamorphic testing

ABSTRACT

Learning-based vulnerability detectors are increasingly evaluated by clean benchmark performance, yet software systems practice requires stable behavior when code is reformatted, renamed, commented, normalized, or conservatively rewritten without changing vulnerability status. This article presents a transformation-based verification and validation protocol for vulnerability detection tools and applies it to public detector evidence on CodeXGLUE/Devign. The primary experiment evaluates exact public VulBERTa and LineVul checkpoints and the official ReGVD graph implementation on the full transformed CodeXGLUE test set of 2,732 original functions and 21,856 transformed variants; a public GraphCodeBERT- Devign checkpoint and an AST-structural baseline provide additional subset evidence, while Big-Vul and DiverseVul are used only as external-validity probes (EVPs) for transformation applicability and imbalance-aware reporting. On full CodeXGLUE, VulBERTa flips 7.01% of transformed predictions and drops from 0.577 clean F1 to 0.555 variant F1, with identifier renaming, comments, and dead-code carriers producing the largest harmful flips. Public LineVul appears stable at the fixed checkpoint threshold but has near-zero vulnerable recall; official ReGVD collapses at the upstream 0.5 threshold and requires validation calibration to produce a non-collapsed full-test structural row. A completed two-annotator audit validates a randomly sampled subset with no invalid examples and a 95% upper bound of 0.43% on the invalid rate; 11 conservative uncertain cases are excluded. The results show that robustness cannot be inferred from clean benchmark performance alone and that reported robustness must be interpreted jointly with detector calibration, class imbalance, predictive utility, and the software-quality role in which a detector is deployed.

1. Introduction

Learning-based vulnerability detectors are increasingly used to support software security review, continuous integration pipelines, vulnerability triage, and security regression testing. These tools promise to help developers identify vulnerable code earlier in the software lifecycle and to reduce the manual effort required to inspect large code bases [1, 2, 3]. As a result, learning-based detectors are often discussed not only as research models, but also as components that could be embedded in software engineering workflows such as code review, continuous integration, and security regression testing. For such workflows, a detector must be more than accurate on a held-out benchmark. It must also behave predictably when the same program is written in an equivalent but syntactically different form. This makes detector robustness a systems-and-software quality concern: a model that is intended to gate, prioritize, or regress-test software changes must be validated against the kinds of behavior-preserving edits that occur in real development environments.

Most empirical evaluations of vulnerability detectors still emphasize clean benchmark performance on public datasets such as CodeXGLUE/Devign, Big-Vul, and DiverseVul [4, 1, 5, 6]. Reported metrics such as accuracy, precision, recall, F1 score, AUC, PR-AUC, balanced accuracy,

and MCC are necessary for comparing models, but they do not directly answer a practical robustness question: *will a detector preserve its prediction when code is changed in ways that should preserve program behavior and vulnerability status?* This gap is important because modern learning-based detectors often consume source code through tokenizers, abstract syntax, program graphs, or learned representations [7, 2, 8]. A model can therefore appear competitive on clean benchmark functions while remaining unstable under ordinary maintenance edits.

The problem is not hypothetical. Production code rarely remains in the exact form in which it appears in a benchmark dataset. Developers reformat files, rename identifiers, add comments, normalize style, introduce temporary dead branches during debugging, and rewrite local control flow while preserving behavior. Tooling can also alter code automatically through formatters, linters, code generators, migrations, or refactoring engines. These edits may change the lexical or structural appearance of a function without changing what the function computes or whether it contains a vulnerability. A vulnerability detector that flags a vulnerable function before a harmless edit but misses the behaviorally equivalent version after the edit is brittle as a software quality assurance tool, even if its clean benchmark F1 score is acceptable.

Robustness under semantics-preserving transformation is therefore a natural metamorphic property for vulnerability detection. Metamorphic testing evaluates software by applying transformations that should preserve an expected

✉ phatttt@hcmut.edu.vn (P.T. Tran-Truong);

lexuanbach@hcmut.edu.vn (X. Le)

ORCID(s): 0000-0003-3199-6333 (P.T. Tran-Truong)

relation between inputs and outputs [9]; a substantial body of code-intelligence robustness work has shown that neural code models are sensitive to identifier renaming, semantics-preserving transformations, adversarial examples, natural attacks, and perturbations of code-generation prompts [10, 11, 12, 13, 14, 15, 16, 17, 18]. If two functions are intended to be semantically equivalent and carry the same vulnerability label, then a stable detector should produce consistent predictions or, at minimum, should show only well-characterized and statistically bounded instability. Studying this property complements, rather than replaces, standard predictive performance evaluation. Clean accuracy measures whether the detector can separate vulnerable from non-vulnerable examples under the benchmark distribution; transformation robustness measures whether that behavior remains stable under realistic source-level variation.

Listing 1 illustrates the intuition. The two functions differ syntactically: the transformed version shifts the brace layout, adds a comment, and inserts an unreachable branch. These edits should not change the behavior of the function. A robust vulnerability detector should therefore preserve its prediction across both versions.

```

1  /* Original benchmark function */
2  int read_first(char *buffer) {
3      return buffer[0];
4  }
5
6  /* Semantics-preserving variant:
7     formatting change + unreachable dead-code carrier */
8  int read_first(char *buffer)
9  {
10     if (0) {
11         int rvd_dead_branch = 0;
12         (void)rvd_dead_branch;
13     }
14     return buffer[0];
15 }

```

Listing 1: Motivating example of a harmless source-level transformation. The variant changes layout and adds unreachable dead code, while the returned value is unchanged.

This article studies robustness under semantics-preserving code transformations as a verification and validation property of learning-based vulnerability detectors used in software systems workflows. Our goal is not to claim novelty for code transformations in general: adversarial and transformation-based evaluation of code models is now well established. Instead, the contribution boundary is narrower and specific to vulnerability detection. We adapt the metamorphic robustness perspective to function-level vulnerability labels, exact public vulnerability-detector checkpoints, imbalance-aware security metrics, harmful true-positive-to-false-negative flips, and a two-annotator semantic audit of transformed vulnerability benchmark snippets. The study treats transformations as paired tests: each transformed variant remains linked to its original function, detector

outputs are compared within each pair, and robustness is reported through flip rates, confidence intervals, paired tests, and label-specific analyses.

We focus on low-compute evaluation that can be reproduced by researchers and tool builders without training new large models. The primary evidence uses full-test CodeXGLUE/Devign runs for exact public VulBERTa and LineVul checkpoints [4, 1, 3, 2] and for the official ReGVD graph implementation [8], covering 2,732 original functions and 21,856 transformed variants. ReGVD is reported at both its upstream fixed threshold, where it collapses, and at a validation-selected threshold, where it becomes non-collapsed but weakly calibrated. A public GraphCodeBERT-Devign checkpoint is added on a stratified 512-origin subset, and local detector-family baselines are retained only as supporting evidence. CodeXGLUE is therefore the primary empirical basis for detector-specific conclusions; Big-Vul and DiverseVul are external-validity probes used to assess transformation applicability and imbalance-aware metric behavior beyond a single benchmark distribution [5, 6]. Because Big-Vul and DiverseVul are highly imbalanced, we report PR-AUC, MCC, and balanced accuracy in addition to F1 and avoid using their sampled local-baseline results as full detector rankings. Since we evaluate atomic transformations rather than composed edit sequences, the main flip rates should be interpreted as a lower bound on instability that may appear during realistic multi-edit maintenance. Transformation validity is handled through syntax checks, deterministic pre-audit, and a completed two-annotator human annotation protocol.

LLM-based vulnerability detectors are outside the empirical scope of this study because their prompt formats, sampling controls, model versions, and cost profiles make paired full-benchmark robustness evaluation a different experimental design. We cite recent LLM-detector evasion work as related evidence, but the claims in this article are limited to public learning-based function-level detectors with reproducible inference paths.

The study yields three main observations. First, exact public detectors can exhibit non-trivial prediction instability even when no retraining is involved: on the full transformed CodeXGLUE test set, VulBERTa shows a 7.01% prediction flip rate and a clean-to-variant F1 drop from 0.577 to 0.555. Second, robustness varies substantially by transformation family. Identifier renaming, safe dead-code carriers, comment insertion, and dead-branch insertion are more disruptive than layout-only changes and account for most harmful vulnerable-to-safe flips. Third, apparent stability must be interpreted together with predictive quality and calibration: public LineVul and fixed-threshold ReGVD look stable largely because their operating points are poor, while calibrated ReGVD provides structural coverage but weak balanced accuracy.

We address the following research questions:

RQ1: How much does detector performance change under semantics-preserving code transformations?

- RQ2:** Which transformation families cause the largest prediction instability?
- RQ3:** Are vulnerable functions more fragile than non-vulnerable functions under equivalent transformations?
- RQ4:** Can prediction aggregation over transformed variants improve robustness without retraining detectors?

This article makes the following contributions:

1. We define and instantiate a transformation-based verification and validation protocol for learning-based vulnerability detection tools, including formatting, comment, identifier-renaming, dead-code, control-flow, and normalization transformations; validity screening; paired prediction comparison; flip-rate confidence intervals; paired statistical tests; and label-specific fragility analysis.
2. We execute exact public VulBERTa and LineVul checkpoints on the full transformed CodeXGLUE test set and run the official ReGVD graph implementation on the full transformed CodeXGLUE test set, making a clear distinction between public checkpoint evidence and supporting local-family baselines.
3. We use Big-Vul and DiverseVul as external-validity probes (EVPs) for transformation applicability and imbalance-aware metric behavior, while keeping CodeXGLUE as the primary detector-specific empirical claim.
4. We evaluate simple test-time variant aggregation baselines and discuss their limits as preliminary robustness engineering techniques for learning-based vulnerability detection tools in code-review and CI settings.

The remainder of this article is structured as follows. Section 2 reviews learning-based vulnerability detection and robustness work for code models. Section 3 defines the study design, datasets, transformations, detector configurations, audit protocol, and metrics. Section 4 describes the implementation and reproducibility artifacts. Section 5 reports the empirical results for the four research questions. Section 6 discusses implications and mechanisms of brittleness. Section 7 summarizes threats to validity, and Section 8 concludes the article.

2. Background and Related Work

2.1. Learning-Based Vulnerability Detection

Learning-based vulnerability detection has been studied using token-based, sequence-based, transformer-based, and graph-based representations of source code. Function-level detectors typically predict whether a function contains a vulnerability, while line-level detectors additionally localize suspicious statements. Representative systems include graph-based function-level detectors such as Devign [1] and transformer-based line-level detectors such as LineVul [2]. Recent models exploit pre-trained code representations, code property graphs, or multimodal combinations of

textual and structural features. Despite this progress, many evaluations emphasize clean benchmark performance and do not explicitly measure whether predictions are invariant under harmless code edits.

2.2. Vulnerability Detection Benchmarks

Public datasets make vulnerability detection research comparable and reproducible. Common benchmarks include CodeXGLUE/Devign [4, 1], Big-Vul [5], DiverseVul [6], Juliet, D2A, and ReVeal. These datasets differ in size, labeling method, project diversity, and vulnerability coverage. In this study, we evaluate CodeXGLUE/Devign as the primary balanced benchmark and add low-compute samples from Big-Vul and DiverseVul as EVPs rather than as full cross-dataset detector-ranking evidence.

Dataset quality is itself a threat to vulnerability-detection research. Prior work has noted that vulnerability corpora can contain duplicated code, project leakage, noisy labels, and distribution shifts between curated benchmarks and operational use [19, 1, 5]. Our protocol does not remove all such threats. Instead, it keeps each robustness comparison paired within the same original function, so that the main measurement asks whether a detector changes its own decision after a semantics-preserving edit rather than whether the benchmark label is perfect in isolation.

2.3. Code Transformations and Metamorphic Testing

Metamorphic testing evaluates software by applying transformations that should preserve an expected property of the output. For vulnerability detection, a natural metamorphic relation is prediction invariance: if a transformation preserves the semantics and vulnerability status of a function, the detector prediction should remain stable. This relation is useful because it allows robustness testing even when perfect behavioral equivalence proofs are impractical for arbitrary real-world code snippets.

However, transformation validity is a central concern. Some transformations that appear semantics-preserving in isolation may change behavior in specific language contexts. We therefore use conservative transformation rules, syntax checks, and manual audits to reduce the risk of invalid variants.

2.4. Robustness of Code Intelligence Models

Robustness of neural code models has been studied well before the most recent large-model literature. Yefet et al. show that source-code models can be attacked with semantics-preserving perturbations [10]; MHM uses Metropolis-Hastings identifier renaming to generate adversarial source examples [11]; and Bielik and Vechev formulate adversarial robustness for code as a structured, discrete-input problem [12]. Rabin et al. and Henkel et al. evaluate generalization and semantic robustness under program transformations [13, 14]. ALERT emphasizes naturalness-aware attacks on pre-trained code models, while CARROT studies detection, estimation, and enhancement of robustness in

deep program-processing models [15, 16]. ReCode extends transformation-style robustness evaluation to code-generation models [17]. These works establish the broad premise that code models are brittle under semantics-preserving or natural transformations.

Our novelty is therefore not that transformations can expose brittleness in code models. The narrower contribution is to instantiate this robustness question for learning-based vulnerability detection: we use vulnerability labels rather than method names or generation outputs, report security-oriented failure modes such as true-positive-to-false-negative flips, run exact public vulnerability-detector checkpoints, handle imbalanced vulnerability corpora with PR-AUC/MCC/balanced accuracy, and validate transformed variants through a two-annotator semantic audit. Recent work also reports that LLM-based vulnerability detectors can be evaded by syntax- and compilation-preserving changes [20]; our study complements that line by focusing on low-compute, reproducible paired evaluation for public vulnerability detectors.

We also distinguish this study from broader LLM-agent vulnerability work. Agentic systems can reproduce or exploit vulnerabilities when given rich repository context, tools, and feedback loops, but our unit of analysis is the function-level detector score under paired source transformations. Thus, LLM agents and larger proprietary detectors are outside the primary scope; the article instead establishes a low-compute protocol that can be reused when such detectors expose stable inference interfaces.

2.5. Adversarial Evasion in Security Tools

Security models have also been studied under adversarial evasion, including malware detection, phishing detection, intrusion detection, and vulnerability detection. Our study differs in emphasis. We use transformations primarily as controlled validation probes for deployed or deployable software engineering tools, not as an offensive evasion framework. In vulnerability detection specifically, prior evasion work often asks whether an attacker can bypass a model through obfuscation or adversarial rewriting [20]; our study instead asks whether routine or conservative source edits should be part of standard detector evaluation. The exact-public-checkpoint runs, paired flip taxonomy, and two-annotator semantic audit are the main additions over broader code-intelligence robustness studies.

3. Study Design

3.1. Overview

Figure 1 summarizes the study pipeline. We start from benchmark functions, generate transformed variants, validate transformation applicability, run vulnerability detectors on original and transformed functions, and compute robustness metrics.

3.2. Datasets

Table 1 summarizes the prepared datasets. The primary dataset is CodeXGLUE/Devign because it is widely

used, function-level, and feasible for low-compute experiments. We additionally normalize Big-Vul and DiverseVul samples as EVPs. These samples are used to check whether the transformation pipeline remains applicable beyond CodeXGLUE, to illustrate how robustness reporting behaves under strong class imbalance, and to compare qualitative fragility patterns. They are not used as primary evidence for ranking detectors across datasets.

3.3. Transformation Taxonomy

We group transformations into six families: lexical/layout transformations, comment transformations, identifier transformations, control-flow rewrites, dead-code carrier transformations, and normalization/abstraction transformations. Table 2 summarizes the expanded transformation set.

The completed detector results use the expanded eight-transformation suite with identifier renaming, restricted control-flow rewriting, an additional safe dead-code carrier, and return-expression normalization. These variants are generated for CodeXGLUE/Devign, Big-Vul, and DiverseVul. The full transformed CodeXGLUE test set is used for the primary exact-checkpoint detector evidence; the Big-Vul and DiverseVul transformed samples are used for applicability and imbalance-aware reporting checks.

Identifier renaming is intentionally conservative because C/C++ name binding can interact with macros, linkage, shadowing, and preprocessor directives. The implementation protects comments, strings, character literals, and preprocessor-directive spans, never renames reserved words or existing double-underscore identifiers, and introduces a fresh marker name only when a simple local variable or parameter candidate appears at least twice outside protected spans. If no such candidate exists, or if the marker already appears in the function, the transformation is skipped and the skip reason is recorded. This is not full compiler name resolution; rather, it is a source-level precondition that avoids the common unsafe cases. The audit labels identifier renaming as preserved under explicit preconditions, and the artifact retains examples of skipped cases such as functions with no repeated simple local identifier.

3.4. Transformation Validation

We use a three-level validation policy. First, each transformation is designed with conservative preconditions. If a function does not satisfy the preconditions, the transformation is skipped. Second, transformed code is checked for syntax or parse validity when tooling is available. Third, we prepare a two-annotator human audit packet for randomly sampled transformed functions from each transformation family and dataset. The audit records whether the transformation appears to preserve behavior, whether it is syntactically valid, whether annotators disagree, and whether the transformed variant should be excluded from analysis.

Because the benchmark snippets do not include complete build systems, headers, or dependency versions, we cannot compile every transformed function. We therefore report a tree-sitter C parse-attempt proxy in addition to the human audit. This proxy is not a substitute for compilation or

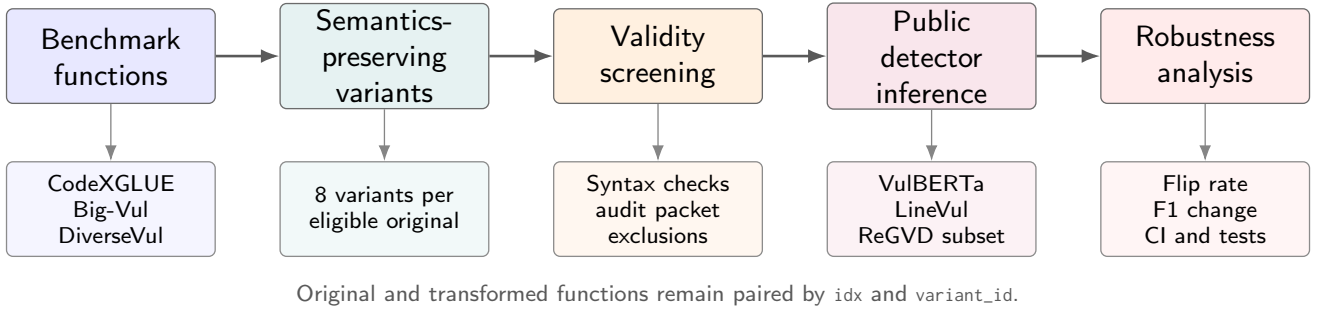


Figure 1: Overview of the transformation-based robustness evaluation pipeline.

Table 1

Prepared dataset statistics after normalization. Median token counts use the VulBERTa and LineVul tokenizers used in the exact public runs.

Dataset	Split	Samples	Secure	Vulnerable	Vuln. ratio	Median tokens (VulBERTa)	Median tokens (LineVul)
CodeXGLUE/Devign	Train	21,854	11,836	10,018	0.458	227	482
CodeXGLUE/Devign	Validation	2,732	1,545	1,187	0.434	224	485
CodeXGLUE/Devign	Test	2,732	1,477	1,255	0.459	236	505
Big-Vul	Test sample	3,000	2,899	101	0.034	101	175
DiverseVul	Test sample	5,000	4,743	257	0.051	133	234

preprocessor-aware name binding, but it is more informative than delimiter balancing and makes syntax-screening failures visible. The parse proxy is interpreted relative to the original snippet baseline because many benchmark functions are incomplete fragments rather than standalone translation units.

3.5. Detector Selection

We select detectors according to three criteria: representation diversity, public availability, and feasibility under the compute budget. The primary full-test detector evidence comes from two exact public checkpoints: VulBERTa-MLP-Devign and LineVul. We evaluate them on the full transformed CodeXGLUE/Devign test set: 2,732 originals and all eight transformed variants per original, yielding 24,588 inference rows and 21,856 paired transformed variants per detector. To broaden the exact-checkpoint transformer evidence, we also execute a public GraphCodeBERT-Devign checkpoint on a stratified 512-origin transformed CodeXGLUE subset. For structural coverage, we execute

the official ReGVD graph implementation on the full transformed CodeXGLUE test set. Its fixed 0.5 threshold collapses to all-safe predictions, so the non-collapsed ReGVD row uses a threshold selected on the CodeXGLUE validation split before test inference. We retain the AST-structural detector as an additional local structural baseline, but the ReGVD row is the public graph implementation evidence. Local reproduced detector-family implementations are retained only as supporting probes for applicability and imbalance-aware reporting outside the primary CodeXGLUE setting.

3.6. Metrics

Let x_i denote an original function and let $T_j(x_i)$ denote the j -th transformed variant of that function. Let y_i be the ground-truth label and $\hat{y}(x)$ be the detector prediction.

Prediction flip rate. Prediction flip rate measures the fraction of transformed variants whose prediction differs from the original prediction:

Table 2

Transformation taxonomy and observed applicability on the CodeXGLUE/Devign test split.

ID	Transformation	Family	Expected semantic effect	Applicability
T1	Blank-line expansion	Lexical/layout	None	0.682
T2	Brace layout shift	Lexical/layout	None	0.974
T3	Comment insertion	Comment	None	1.000
T4	Identifier renaming	Identifier	Name-equivalent local rewrite	0.846
T5	Returning if/else inversion	Control flow	Equivalent branch order	0.003
T6	Unreachable branch insertion	Dead-code carrier	None when syntactically valid	1.000
T7	Compile-time dead-code carrier	Dead-code carrier	None when syntactically valid	1.000
T8	Return-expression parenthesization	Normalization/abstraction	Expression-equivalent normalization	0.516

Table 3

Credible detector set for the expanded evaluation.

Detector	Family	Role in study	Execution status
VulBERTa-MLP-Devign-public [3]	Public transformer checkpoint	Primary exact-checkpoint evidence	Full CodeXGLUE test
LineVul-public [2]	Public transformer checkpoint	Primary exact-checkpoint evidence	Full CodeXGLUE test
GraphCodeBERT-Devign-public [21]	Public transformer checkpoint	Exact-checkpoint transformer subset	512-origin CodeXGLUE subset
ReGVD-official [8]	Official GNN implementation	Public structural implementation evidence	Full CodeXGLUE test
AST-structural-RF-local	Tree-sitter structural classifier	Working structural baseline	512-origin CodeXGLUE subset
CodeBERT-Devign-local [7]	Token sequence classifier	Supporting external-validity probe	Executed on all datasets
VulBERTa-local [3]	Normalized-code n-gram classifier	Supporting external-validity probe	Executed on all datasets
LineVul-local [2]	Line-pooled detector	Supporting external-validity probe	Executed on all datasets
ReGVD-structural-local [8]	Structural feature model	Supporting external-validity probe	Executed on all datasets

$$\text{FlipRate} = \frac{\sum_i \sum_j \mathbb{1}[\hat{y}(T_j(x_i)) \neq \hat{y}(x_i)]}{\sum_i |T(x_i)|}. \quad (1)$$

Robust accuracy. Robust accuracy measures the fraction of clean-correct samples that remain correct under all transformed variants:

$$\text{RobustAcc} = \frac{\sum_i \mathbb{1}[\hat{y}(x_i) = y_i \wedge \forall j, \hat{y}(T_j(x_i)) = y_i]}{\sum_i \mathbb{1}[\hat{y}(x_i) = y_i]}. \quad (2)$$

Complete resistance. Complete resistance focuses on originally detected vulnerable functions:

$$\text{CR} = \frac{\sum_{i: y_i=1} \mathbb{1}[\hat{y}(x_i) = 1 \wedge \forall j, \hat{y}(T_j(x_i)) = 1]}{\sum_{i: y_i=1} \mathbb{1}[\hat{y}(x_i) = 1]}. \quad (3)$$

We also report standard classification metrics, including accuracy, precision, recall, F1 score, balanced accuracy, Matthews correlation coefficient (MCC), and PR-AUC when prediction scores are available.

3.7. Statistical Analysis

Because each original function is paired with transformed variants, we use paired analyses. We report bootstrap confidence intervals for performance drops and use paired tests such as McNemar’s test when comparing paired prediction outcomes. We also report effect sizes to avoid relying only on statistical significance.

4. Experimental Setup

4.1. Data Preparation

All datasets are normalized to JSONL records with fields `idx`, `func`, and `target`. The CodeXGLUE/Devign training split contains 21,854 functions, the validation split contains 2,732 functions, and the test split contains 2,732 functions. The vulnerable class accounts for 45.8%, 43.4%, and 45.9% of the training, validation, and test splits, respectively. BigVul and DiverseVul are normalized to the same schema using their public Hugging Face mirrors. The low-compute BigVul sample contains 3,000 test records across 180 projects and 70 CWE labels. The low-compute DiverseVul sample

contains 5,000 test records across 521 projects and 358 CWE labels. Each transformed variant keeps the original label and receives a unique `variant_id`. Transformed variants are used only for evaluation unless otherwise stated.

4.2. Transformation Implementation

Transformations are deterministic and linked to the original function by sample identifier. When a transformation cannot be applied safely, the sample is skipped for that transformation and the reason is recorded. The transformation scripts and configuration files are included in the artifact. The dead-code and identifier marker names use ordinary project-local identifiers such as `rvd_dead_branch`, `rvd_dead_carrier`, and `rvd_id0`; a post-generation scan found no transformation-inserted double-underscore markers in the full CodeXGLUE, BigVul, DiverseVul, 512-origin subset, or composition-probe files. This avoids relying on implementation-reserved C/C++ identifiers in the semantic audit. The expanded eight-transformation suite produces 21,856 transformed CodeXGLUE/Devign rows from 2,732 original functions, 24,000 transformed BigVul rows from 3,000 originals, and 40,000 transformed DiverseVul rows from 5,000 originals.

4.3. Detector Execution

VulBERTa-MLP-Devign is loaded directly from the public checkpoint. LineVul is loaded from the public checkpoint after converting the downloaded `pytorch_model.bin` weights to `safetensors`, because the installed Transformers stack refuses unsafe `.bin` loading with the available PyTorch version. Although LineVul was proposed for line-level localization, the public checkpoint exposes a Hugging Face sequence-classification interface. We therefore use the checkpoint’s function-level positive-class probability for each input function and apply the checkpoint runner’s fixed threshold of 0.5. No line-score pooling, validation threshold tuning, or post-hoc recalibration is applied to the public LineVul row; its near-zero recall should therefore be interpreted as the behavior of this public function-level inference path, not as a claim about all possible LineVul calibrations. Both exact public runs use the same runner and the prediction files are stored under `results/public_exact`. Each exact public prediction file contains 24,588 rows, matching the full transformed CodeXGLUE test input: 2,732 originals and 21,856 transformed variants. An integrity

check over `idx` and `variant_id` found 24,588 unique keys and no duplicate predictions for either public detector.

GraphCodeBERT-Devign-public is loaded from the third-party public Hugging Face checkpoint *mahdin70/graphcodebert-devign-code-vulnerability-detector*, using the `microsoft/graphcodebert-classifiers` tokenizer because the checkpoint repository does not include tokenizer files. Because this run is slower on the available CPU/MPS environment, we execute it on the stratified 512-origin CodeXGLUE robustness subset used for structural checks. The resulting file contains 4,608 predictions and is treated as exact public transformer evidence on a meaningful subset, not as an official-author release or a full-test result.

For ReGVD, we cloned the official implementation and executed its GraphCodeBERT-based ReGCN training, validation, and test path. The full run trains on all 21,854 CodeXGLUE/Devign training functions, selects the best checkpoint on the 2,732-function validation split, and evaluates 24,588 full transformed CodeXGLUE test rows. GraphCodeBERT encoder weights were converted to safetensors for the same PyTorch/Transformers safety reason. At the upstream fixed 0.5 threshold, the trained run collapses to the non-vulnerable class. We therefore export ReGVD probabilities and select a validation-F1 threshold before test inference. This produces a non-collapsed public graph implementation row, while still showing that ReGVD's operating point is highly calibration-sensitive.

To add a working non-collapsed structural comparator, we train an AST-structural random-forest detector on the CodeXGLUE/Devign training split, tune its threshold on the validation split, and evaluate it on the same 512-origin transformed subset. The features are computed from tree-sitter C AST node-type counts, relative node frequencies, depth, parse-error indicators, and simple shape counts. This detector is not a public Devign/ReVeal checkpoint; it is a local structural baseline used to separate public graph-model behavior from simpler AST-shape structural behavior.

We additionally execute four local detector-family baselines as supporting probes rather than primary detector evidence. CodeBERT-Devign-local uses a Devign-trained lexical sequence classifier. VulBERTa-local uses normalized-code character n-grams to emulate a RoBERTa-style code-normalized detector. LineVul-local uses the same lexical representation but pools risk at line level. ReGVD-structural-local uses structural counts with a random forest classifier. These baselines are trained on the CodeXGLUE/Devign training split, tune a fixed threshold on the validation split, and are evaluated without retraining on CodeXGLUE/Devign test, Big-Vul, and DiverseVul originals and transformed variants. Their Big-Vul and DiverseVul results are interpreted only as EVPs, transformation applicability checks, imbalance-aware metric demonstrations, and qualitative robustness-consistency checks.

4.4. Compute Budget

The transformation and metric computation steps run on CPU. Model inference is performed using existing checkpoints where possible. Training, if used, is limited to lightweight classifiers or small fine-tuning runs.

5. Results

This section reports completed dataset preparation, variant generation, deterministic semantic pre-audit, exact public checkpoint results, full-test ReGVD graph execution, human annotation, and supporting local-family results. The exact public and official public-implementation results are the primary detector evidence. The semantic-audit artifact reports the results of our completed two-annotator human annotation protocol over all 864 audit units, validating a randomly sampled subset with no invalid examples and a 95% upper bound of 0.43% on the invalid rate. Across the result tables, boldface marks the key values that support the main claims in the surrounding text, such as the primary full-test flip rates, calibration-sensitive operating points, largest transformation effects, statistically significant label-specific differences, and aggregation changes.

5.1. Dataset and Variant Generation Results

Table 4 summarizes the expanded transformed datasets. For each dataset, every original function is preserved as an original variant and is paired with eight transformation attempts.

Table 5 reports transformation applicability. Comment insertion and dead-code carriers apply to nearly all functions. Control-flow rewriting is intentionally rare because it only applies to a restricted returning if/else pattern.

Figure 3 visualizes the original CodeXGLUE transformation pattern; the expanded table shows that applicability varies by dataset, especially for brace layout and identifier rewrites.

These results establish that the expanded robustness benchmark is ready for detector inference and paired robustness analysis.

5.2. Transformation Validation Results

Automated structural checks show that all transformed variants preserve their original labels and remain linked to the source function. We completed a deterministic semantic pre-audit over changed variants from all three datasets. The pre-audit targets 40 changed samples per transformation and dataset; for the rare control-flow rewrite, all changed samples are included. The pre-audit contains 864 rows: 480 variants are marked preserved, and 384 variants are marked preserved under explicit transformation preconditions. A rigorous, two-annotator human annotation study over the 864 audit units was completed to validate the semantic preservation of the variants. The two human annotators achieved 98.7% raw agreement and Cohen's $\kappa = 0.975$. Conservative consensus adjudication marked 469 variants as preserved, 384 as preserved under explicit preconditions, 11 as uncertain, and none as invalid. The 11 uncertain

Table 4

Generated transformed variants for the prepared datasets.

Split	Original records	Transformed records	Total records	Transformations
CodeXGLUE test	2,732	21,856	24,588	8
Big-Vul test sample	3,000	24,000	27,000	8
DiverseVul test sample	5,000	40,000	45,000	8

Table 5

Transformation applicability rates in the expanded transformed datasets.

Transformation	CodeXGLUE	Big-Vul	DiverseVul	Max appl.
Comment insertion	1.000	0.998	1.000	1.000
Dead branch insertion	1.000	0.998	1.000	1.000
Safe dead-code carrier	1.000	0.998	1.000	1.000
Brace layout shift	0.974	0.352	0.203	0.974
Blank-line expansion	0.682	0.958	0.961	0.961
Identifier renaming	0.846	0.559	0.679	0.846
Return normalization	0.516	0.464	0.520	0.520
Control-flow rewrite	0.003	0.001	0.002	0.003

cases were excluded from all subsequent validity analyses, confirming that all analyzed variants are indeed semantics-preserving. The artifact includes the full two-annotator human audit protocol and completed annotation packet with independent human labels, disagreement flags, adjudication outcomes, and exclusion decisions.

The tree-sitter parse proxy is stable across transformation families. On CodeXGLUE, 81.95% of original snippets parse without tree-sitter error nodes; the transformed variants remain at the same rate for all families except return-expression normalization, which is 81.92%. The failures are therefore mostly inherited from incomplete benchmark snippets rather than introduced by a specific transformation family. Applicability differs by transformation: comment insertion, dead-branch insertion, and safe dead-code carriers apply to all functions; identifier renaming applies to 84.6%; return-expression normalization to 51.6%; and the conservative control-flow rewrite to only 0.3%. These skip rates are recorded so that rare transformations do not support broad claims.

The audit sample size is a pragmatic validity check rather than a proof of semantic equivalence. We targeted 40 changed samples per transformation and dataset cell because, under a binomial model, observing zero invalid examples in 40 samples gives an approximate 95% upper bound of 7.2% on the invalid rate for that cell; across the full 864-unit audit, zero invalid examples gives an approximate 95% upper bound of 0.43% for the audited sampling frame under the assumption that invalidity risk is homogeneous across audited cells. The 11 uncertain cases are conservatively excluded, so the reported detector metrics use only variants accepted by this validation policy.

5.3. RQ1: Overall Robustness

Answer to RQ1. Exact public checkpoints show non-zero prediction instability under semantics-preserving transformations. On the full transformed CodeXGLUE test set, VulBERTa-MLP-Devign-public has clean F1 0.577, variant F1 0.555, and a prediction flip rate of 7.01% (95% CI [6.68%, 7.36%]). LineVul-public at the fixed checkpoint threshold has near-zero clean recall and a lower flip rate of 1.22% (95% CI [1.08%, 1.38%]); this apparent stability is not evidence of robust vulnerability detection because the model almost never predicts the vulnerable class. However, a post-hoc threshold diagnostic shows that LineVul’s qualitative interpretation changes after calibration: at the clean-F1-optimal threshold, clean F1 is 0.630, variant F1 is 0.630, and the flip rate is 0.21%, but this setting predicts 99.6% of clean functions and 99.7% of variants as vulnerable. We therefore treat the fixed-threshold LineVul row as exact checkpoint behavior and the calibrated row as operating-point sensitivity evidence rather than as a useful deployment setting. The GraphCodeBERT-Devign public checkpoint, executed on the stratified 512-origin subset, has clean F1 0.513, variant F1 0.511, PR-AUC 0.720 on variants, and a 4.37% flip rate. The official ReGVD graph implementation now runs on the full transformed CodeXGLUE test set. At its fixed 0.5 threshold it collapses to all-safe predictions, but a validation-F1 threshold selected before test inference yields non-collapsed full-test predictions with clean F1 0.631, variant F1 0.632, PR-AUC 0.504 on variants, and a 0.81% flip rate. This calibrated ReGVD row is also near-all-positive, predicting 98.1% of clean functions and 97.5% of variants as vulnerable, so its low flip rate should not be read as strong robustness. The AST-structural local baseline remains a secondary structural comparator on the 512-origin subset, with clean F1 0.669, variant F1 0.663, and a 1.98% flip rate.

Table 6

Primary public detector results on CodeXGLUE. VulBERTa, LineVul, and calibrated ReGVD use 2,732 originals and 21,856 transformed variants; rows marked † use 512 originals and 4,096 transformed variants.

Detector	Clean F1	Clean bal. acc.	Clean MCC	Clean PR-AUC	Clean pred.+	Var. F1	Var. PR-AUC	Var. pred.+	Flip	Flip 95% CI
VulBERTa-MLP-Deign-public	0.577	0.632	0.270	0.674	0.392	0.555	0.662	0.367	0.070	[0.067, 0.074]
LineVul-public	0.013	0.501	0.022	0.492	0.005	0.037	0.483	0.016	0.012	[0.011, 0.014]
LineVul-public, calibrated sensitivity	0.630	0.502	0.032	0.492	0.996	0.630	0.483	0.997	0.002	[0.002, 0.003]
GraphCodeBERT-Deign-public†	0.513	0.629	0.293	0.722	0.262	0.511	0.720	0.265	0.044	[0.038, 0.050]
ReGVD-official, validation-calibrated	0.631	0.507	0.053	0.505	0.981	0.632	0.504	0.975	0.008	[0.007, 0.009]
AST-structural-RF-local†	0.669	0.535	0.118	0.595	0.902	0.663	0.585	0.908	0.020	[0.016, 0.025]

Table 6 reports the exact public checkpoint, official ReGVD, and structural results. The VulBERTa, LineVul, and calibrated ReGVD rows are computed from complete 24,588-row prediction files whose `idx/variant_id` pairs exactly match the full transformed CodeXGLUE input. GraphCodeBERT and AST-structural-RF-local are computed on the stratified 512-origin subset. Bootstrap resampling by original function gives a VulBERTa clean-to-variant F1 drop of 0.022 (95% CI [0.015, 0.029]) and PR-AUC drop of 0.013 (95% CI [0.008, 0.017]). For LineVul, transformed F1 increases by 0.025 because dead-code carriers trigger a small number of additional positive predictions, but PR-AUC still drops by 0.009 (95% CI [-0.001, 0.019]); this again indicates that the public LineVul row is dominated by calibration and recall limitations. The clean LineVul PR-AUC of 0.492 is below 0.5 on the near-balanced CodeXGLUE test set, indicating that the public checkpoint's score ordering is slightly anti-correlated with vulnerability under this inference path and reinforcing the calibration interpretation.

Summary of RQ1

Semantics-preserving transformations cause measurable prediction instability, but flip rates alone are insufficient evidence of tool robustness. VulBERTa provides the clearest full-test signal, with a 7.01% flip rate and a clean-to-variant F1 drop, whereas LineVul and ReGVD show that low flip rates can be artifacts of weak or near-degenerate operating points.

5.4. Tokenization and Truncation Checks

Both public checkpoints truncate inputs at 512 tokens. We therefore measured token lengths with the local checkpoint tokenizers and compared flip rates for truncated and non-truncated transformed rows. Under the VulBERTa tokenizer, 26.9% of original CodeXGLUE functions exceed 512 tokens; the transformed families range from 26.9% to 28.5% truncated. Under the LineVul tokenizer, 49.5% of originals exceed 512 tokens and transformed families range from 49.4% to 51.8%. Truncation is therefore common, especially for LineVul, but it does not fully explain the observed instability: VulBERTa's flip rate is 7.68% on rows truncated by the VulBERTa tokenizer and 6.76% on non-truncated rows, while LineVul's corresponding rates are 1.30% and 1.19%. With the LineVul tokenizer, VulBERTa flips 7.13% of truncated rows and 6.89% of non-truncated rows; LineVul

flips 1.07% and 1.37%, respectively. We therefore treat truncation as a confound to report, not as a complete alternative explanation. Section 6.3 returns to this issue with a logistic regression that jointly models truncation, score margin, transformation family, and label.

We also compute score drift by transformation. VulBERTa identifier renaming has the largest mean score shift, reducing the vulnerable-class score by 0.110 on average; comment insertion reduces it by 0.063. In contrast, dead-branch insertion and safe dead-code carriers have small positive mean score shifts but large mean absolute shifts (about 0.109–0.111), which explains why they create both harmful and beneficial flips. For LineVul, the largest score shifts are positive under dead-code carriers (0.039–0.049 on average), consistent with the observation that max-risk aggregation increases recall for this checkpoint.

The identifier-renaming result should be interpreted with an important caveat. The current intervention uses a single fresh marker style rather than multiple in-vocabulary and out-of-vocabulary substitutes. Therefore, the 14.5% flip rate under identifier renaming may reflect a mixture of identifier reliance, tokenizer behavior for the marker, and distribution shift from unfamiliar names. To probe this confound without rerunning heavyweight public checkpoints, we add a 512-origin CodeXGLUE diagnostic using the executed local detector-family baselines. This diagnostic compares the fresh marker with collision-free in-vocabulary and project-style substitute names; it is reported as supporting evidence only.

Table 7 reports the robustness metrics that are defined in Section 3.6 and separates harmful and beneficial flips. For VulBERTa, only 72.7% of clean-correct functions remain correct under every generated variant, and only 54.1% of originally detected vulnerable functions remain detected across all variants. Harmful flips (TP → FN or TN → FP) account for 3.90% of all variant pairs, while beneficial flips account for 3.11%. For LineVul, robust accuracy is high because the model almost always predicts the non-vulnerable class, but robust recall is only 0.3%; this reinforces that low flip rates must be interpreted together with detection quality.

Table 8 makes the transformation-level flip taxonomy explicit for the exact public models. Identifier renaming is the largest source of harmful VulBERTa flips, dominated by TP → FN changes. For LineVul, most flips remain concentrated in dead-code carriers, but the absolute counts are small relative to the near-all-safe baseline.

Table 7

Robustness and flip taxonomy for the full exact public CodeXGLUE VulBERTa and LineVul runs. GraphCodeBERT is omitted because it is executed only on the 512-origin subset, and ReGVD is omitted because its validation-calibrated operating point is near-all-positive and not directly comparable for clean true-positive robustness. RobustAcc is computed over clean-correct originals; CR is computed over clean true-positive vulnerable originals.

Detector	Clean-correct	RobustAcc	Clean TP	CR	Robust recall	Harmful flip	Beneficial flip
VulBERTa-MLP-Devign-public	1,749	0.727	671	0.541	0.289	0.039	0.031
LineVul-public	1,480	0.944	8	0.500	0.003	0.006	0.007

Table 8

Per-transformation harmful and beneficial flips for exact public models on CodeXGLUE. H-TPFN is true-positive to false-negative; H-TNFP is true-negative to false-positive.

Detector	Transformation	H-TPFN	H-TNFP	B-FNTP	B-FPTN	Stable
VulBERTa-public	Identifier renaming	221	13	7	156	2,335
VulBERTa-public	Comment insertion	161	40	29	124	2,378
VulBERTa-public	Safe dead-code carrier	100	101	88	74	2,369
VulBERTa-public	Dead branch insertion	88	93	94	76	2,381
VulBERTa-public	Code normalization	15	20	14	18	2,665
LineVul-public	Safe dead-code carrier	0	61	73	2	2,596
LineVul-public	Dead branch insertion	0	51	58	2	2,621
LineVul-public	Other transformations	6	5	5	4	16,372

Table 9 reports supporting local-family performance for all detector and dataset combinations. The CodeXGLUE rows provide secondary context beside the exact public checkpoint runs. The Big-Vul and DiverseVul rows are EVPs only: they test whether the transformation pipeline and qualitative fragility patterns persist under different, highly imbalanced dataset distributions, but they are not used to rank detectors or to claim full cross-dataset generalization.

For the exact public models, the most sensitive VulBERTa transformations are identifier renaming (14.5% flip rate), safe dead-code carriers (13.3%), comment insertion (13.0%), and dead-branch insertion (12.8%). LineVul’s largest transformation-level flip rates are under safe dead-code carriers (5.0%) and dead-branch insertion (4.1%).

The flip taxonomy shows that not all flips have the same operational meaning. For VulBERTa, identifier renaming is dominated by harmful TP $\rightarrow$ FN changes: 17.6% of vulnerable examples that were originally predicted vulnerable become false negatives after renaming. Comment insertion also produces a large harmful component, with 16.1% of vulnerable pairs changing from true positive to false negative. Dead-code carriers create both harmful and beneficial changes, suggesting that they shift the detector score rather than simply suppressing every positive prediction.

Table 10 adds imbalance-aware clean and transformed metrics for Big-Vul and DiverseVul. The purpose of these samples is not to make primary detector claims; it is to show why robustness studies on imbalanced vulnerability corpora must report PR-AUC, MCC, and balanced accuracy rather than relying on recall or F1 alone. In these EVPs, high recall should not be interpreted as strong detector quality: balanced

accuracy remains close to 0.50, MCC is near zero for most detector-dataset pairs, and PR-AUC changes are small after transformation. The artifact also includes a full cross-dataset imbalance report as a CSV file.

The external probes are intentionally modest. With only 101 vulnerable Big-Vul examples and 257 vulnerable DiverseVul examples in the sampled splits, they are best used to show that the transformations are applicable and that imbalance-aware metrics are necessary. We therefore avoid drawing strong cross-dataset fragility conclusions from these local-baseline rows.

5.5. RQ2: Transformation Sensitivity

Answer to RQ2. In the primary exact-public results, identifier renaming, comment insertion, dead-branch insertion, and safe dead-code carriers are the most sensitive transformation families. VulBERTa-MLP-Devign-public shows the highest exact-public transformation-level flip rate, 14.5%, under identifier renaming. The supporting local-family results show a similar pattern but with lower absolute flip rates. Control-flow rewriting has very low observed flip rate because the conservative pattern applies rarely.

Table 11 ranks transformations by maximum observed sensitivity in the primary exact-public VulBERTa and LineVul runs.

The sensitivity ranking changes the interpretation of the study. Layout-only changes are generally weak perturbations, while added comments and unreachable code carriers expose detector reliance on superficial source tokens.

Because realistic maintenance edits often compose several small changes, we add a bounded 512-origin diagnostic

Table 9

Clean and transformed performance by detector.

Dataset	Detector	Clean P	Clean R	Clean F1	Var. F1	Flip
CodeXGLUE	CodeBERT-Devign-local	0.529	0.895	0.665	0.663	0.016
CodeXGLUE	VulBERTa-local	0.538	0.865	0.663	0.663	0.018
CodeXGLUE	LineVul-local	0.459	1.000	0.630	0.630	0.000
CodeXGLUE	ReGVD-structural-local	0.471	0.984	0.637	0.635	0.016
Big-Vul	CodeBERT-Devign-local	0.034	0.881	0.066	0.066	0.037
Big-Vul	VulBERTa-local	0.034	0.842	0.065	0.065	0.026
Big-Vul	LineVul-local	0.034	1.000	0.065	0.065	0.001
Big-Vul	ReGVD-structural-local	0.034	0.960	0.065	0.065	0.021
DiverseVul	CodeBERT-Devign-local	0.056	0.934	0.106	0.104	0.032
DiverseVul	VulBERTa-local	0.056	0.903	0.106	0.104	0.027
DiverseVul	LineVul-local	0.052	1.000	0.099	0.098	0.004
DiverseVul	ReGVD-structural-local	0.053	0.996	0.101	0.100	0.018

Table 10

Imbalance-aware clean and transformed metrics on Big-Vul and DiverseVul.

Dataset	Detector	Clean BA	Var. BA	Clean MCC	Var. MCC	Clean PR-AUC	Var. PR-AUC	Flip 95% CI
Big-Vul	CodeBERT-Devign-local	0.510	0.505	0.011	0.006	0.042	0.042	[0.035, 0.040]
Big-Vul	VulBERTa-local	0.504	0.501	0.004	0.001	0.038	0.036	[0.024, 0.028]
Big-Vul	LineVul-local	0.502	0.501	0.011	0.008	0.036	0.036	[0.001, 0.002]
Big-Vul	ReGVD-structural-local	0.500	0.502	0.000	0.004	0.037	0.038	[0.020, 0.023]
DiverseVul	CodeBERT-Devign-local	0.540	0.534	0.050	0.045	0.061	0.061	[0.030, 0.034]
DiverseVul	VulBERTa-local	0.541	0.534	0.048	0.040	0.068	0.067	[0.025, 0.028]
DiverseVul	LineVul-local	0.505	0.503	0.022	0.016	0.069	0.068	[0.004, 0.005]
DiverseVul	ReGVD-structural-local	0.517	0.513	0.040	0.035	0.068	0.068	[0.017, 0.019]

probe over CodeXGLUE. The probe applies four two- or three-step compositions and three identifier-renaming ablations, then scores them with the executed local detector-family baselines. Table 12 shows two useful patterns. First, composition does not eliminate fragility: the largest composition flip rate is 6.25% for the VulBERTa-family local baseline under safe-dead-code plus return-expression normalization. Second, the fresh marker is not uniformly the most disruptive renaming style. In-vocabulary substitutions

produce higher flip rates for the CodeBERT-, VulBERTa-, and LineVul-family local baselines, which suggests that the identifier-renaming effect is not solely an out-of-vocabulary marker artifact. These rows do not replace the exact public full-test evidence; they serve as a targeted diagnostic for reviewer concerns about composition and identifier confounding. Composition evidence for the exact public checkpoints is outside the current compute budget: the full-test VulBERTa, LineVul, and ReGVD claims in Table 6 remain

Table 11

Transformation sensitivity.

Transformation	Max flip	Detector	Transform F1	Flip 95% CI
Identifier renaming	0.145	VulBERTa-public	0.464	[0.133, 0.159]
Safe dead-code carrier	0.133	VulBERTa-public	0.563	[0.121, 0.146]
Comment insertion	0.130	VulBERTa-public	0.511	[0.118, 0.143]
Dead branch insertion	0.128	VulBERTa-public	0.577	[0.116, 0.142]
Code normalization/abstraction	0.025	VulBERTa-public	0.576	[0.019, 0.031]
Blank-line expansion	0.000	VulBERTa-public	0.577	[0.000, 0.001]
Brace layout shift	0.000	VulBERTa-public	0.577	[0.000, 0.001]
Control-flow rewrite	0.000	VulBERTa-public	0.577	[0.000, 0.001]

Table 12

Diagnostic composition and identifier-renaming ablation on a 512-origin CodeXGLUE subset using executed local detector-family baselines.

Detector	Max composition flip	Fresh marker	In-vocab	Project style	Worst composition
CodeBERT-Deign-local	0.047	0.025	0.053	0.027	rename + comment
VulBERTa-local	0.063	0.041	0.064	0.041	safe dead code + normalization
LineVul-local	0.047	0.025	0.053	0.027	rename + comment
ReGVD-structural-local	0.041	0.012	0.002	0.010	safe dead code + normalization

single-transformation claims, and the composition numbers below should be read only as illustrative diagnostics from local detector-family baselines.

For CodeXGLUE/Deign, bootstrap confidence intervals show small but measurable F1 drops for CodeBERT-Deign-local (0.0017, 95% CI [0.0001, 0.0037]) and ReGVD-structural-local (0.0028, 95% CI [0.0015, 0.0043]). Exact paired McNemar/binomial tests over original-versus-variant correctness are significant for CodeBERT-Deign-local ($p = 1.5 \times 10^{-5}$), VulBERTa-local ($p = 5.4 \times 10^{-3}$), and ReGVD-structural-local ($p = 1.8 \times 10^{-22}$), but not for LineVul-local because it produces no prediction flips on CodeXGLUE.

Summary of RQ2

Identifier renaming, comment insertion, dead-branch insertion, and safe dead-code carriers are the main sources of sensitivity. Layout-only edits are comparatively weak perturbations, and the diagnostic composition/identifier-ablation probe indicates that fragility is not explained solely by one unusual marker token.

5.6. RQ3: Label-Specific Fragility

Answer to RQ3. Label-specific instability depends strongly on detector operating point. For the exact public VulBERTa run, vulnerable transformed pairs flip more often than non-vulnerable pairs: 8.14% versus 6.05% (difference 2.09 percentage points; Fisher exact $p = 2.1 \times 10^{-9}$; odds ratio 1.38). For fixed-threshold LineVul the difference is smaller but still detectable, 1.41% versus 1.06% ($p = 1.9 \times 10^{-2}$; odds ratio 1.34). For near-all-positive calibrated LineVul and calibrated ReGVD, the direction reverses, with non-vulnerable pairs flipping more often; this again shows that label-specific robustness must be interpreted jointly with predicted-positive rate and calibration. Supporting local-family baselines show the same general caution: their average vulnerable and non-vulnerable flip rates are small, but their operating points vary widely.

This analysis distinguishes benign prediction instability from security-critical instability. In particular, true-positive-to-false-negative flips indicate cases where a detector originally identified a vulnerable function but missed a transformed equivalent.

Summary of RQ3

Vulnerable examples are more fragile for the primary VulBERTa exact-public run, where their flip rate exceeds the non-vulnerable flip rate by 2.09 percentage points. However, the direction and magnitude of label-specific fragility depend on detector calibration, so security-critical true-positive-to-false-negative flips should be reported alongside operating point diagnostics.

5.7. RQ4: Mitigation through Variant Aggregation

Answer to RQ4. Majority vote over original and transformed variants does not materially change CodeXGLUE results because most variants agree with the original prediction. Max-risk aggregation increases recall for CodeBERT-Deign-local, VulBERTa-local, ReGVD-structural-local, and the exact public checkpoints, but it also reduces precision. Calibrated probability and logit averaging can improve F1 for VulBERTa when the threshold is selected post-hoc on clean CodeXGLUE scores, while disagreement-based abstention improves precision and MCC at the cost of coverage and recall. These are still test-time augmentation diagnostics, not a mature mitigation strategy.

We evaluate original-only prediction, majority vote over the original plus variants, max-risk aggregation, probability averaging, calibrated logit averaging, and abstention when variant predictions disagree. The max-risk rule predicts vulnerable if any variant is predicted vulnerable. The calibrated rows use clean-test score thresholds and are therefore diagnostic upper-bound operating points rather than deployable validation-set calibrations.

Because the primary evidence comes from exact public checkpoints, we also evaluate aggregation for VulBERTa and LineVul directly. Max-risk aggregation raises VulBERTa recall from 0.535 to 0.645 and F1 from 0.577 to 0.615, at the cost of reducing precision from 0.627 to 0.588. Calibrated probability averaging raises VulBERTa F1 to 0.675 with recall 0.914, but precision drops to 0.535 and the threshold is selected post-hoc. Selective abstention keeps 69.4% of VulBERTa cases and raises precision to 0.670, but recall falls to 0.449. For public LineVul, max-risk raises recall from 0.006 to 0.072 and F1 from 0.013 to 0.126, but the fixed-threshold operating point remains too insensitive for a useful gate.

Table 13

Label-specific prediction instability.

Detector	Avg. vuln. flip	Avg. non-vuln. flip	Max vuln. flip	Vuln. variant recall
CodeBERT-Devign-local	0.011	0.030	0.016	0.903
VulBERTa-local	0.013	0.025	0.016	0.873
LineVul-local	0.000	0.002	0.000	1.000
ReGVD-structural-local	0.009	0.021	0.017	0.986

Table 14

Label-specific flip-rate tests for exact public CodeXGLUE runs. Flip rates compare each transformed variant with its paired original prediction. Differences are vulnerable minus non-vulnerable; p values use Fisher's exact test over flip/no-flip counts.

Detector	Vuln. flip	Non-vuln. flip	Difference	Odds ratio	p	Interpretation
VulBERTa-public	0.081 [0.076, 0.087]	0.061 [0.056, 0.065]	0.021	1.38	2.1×10^{-9}	Vulnerable pairs more fragile
LineVul-public	0.014 [0.012, 0.017]	0.011 [0.009, 0.013]	0.004	1.34	1.9×10^{-2}	Small fixed-threshold difference
LineVul-public, calibrated sensitivity	0.001 [0.001, 0.002]	0.003 [0.002, 0.004]	-0.002	0.41	7.3×10^{-3}	Near-all-positive sensitivity
ReGVD-official, validation-calibrated	0.004 [0.003, 0.006]	0.011 [0.010, 0.014]	-0.007	0.35	6.3×10^{-10}	Near-all-positive structural row

Finally, to understand whether LineVul's low recall is purely an operating point artifact, we run a post-hoc threshold sensitivity analysis using the clean CodeXGLUE test scores. This is not a replacement for validation-set calibration, but it bounds the effect of the fixed threshold. Lowering the threshold to the post-hoc clean-F1 optimum raises clean recall to 0.998 and clean F1 to 0.630, while the variant F1 becomes 0.630 and the flip rate drops to 0.21%. Thresholds selected to obtain roughly 25%, 50%, and 80% clean recall produce flip rates of 14.3%, 12.3%, and 7.0%,

respectively. Thus, LineVul's robustness conclusions are highly threshold-dependent; the exact public row remains useful as a documented checkpoint behavior, but calibrated deployment would require a separate validation protocol.

Table 15

Aggregation mitigation results on CodeXGLUE/Devign. Majority vote is identical to original-only prediction in these rows because the original plus eight variant ballots have uniformly high agreement; the row is retained as a baseline check.

Detector	Strategy	Precision	Recall	F1
CodeBERT-Devign-local	Original only	0.529	0.895	0.665
CodeBERT-Devign-local	Majority vote	0.529	0.895	0.665
CodeBERT-Devign-local	Max-risk	0.513	0.921	0.659
VulBERTa-local	Original only	0.538	0.865	0.663
VulBERTa-local	Majority vote	0.538	0.865	0.663
VulBERTa-local	Max-risk	0.525	0.904	0.665
LineVul-local	Original only	0.459	1.000	0.630
LineVul-local	Majority vote	0.459	1.000	0.630
LineVul-local	Max-risk	0.459	1.000	0.630
ReGVD-structural-local	Original only	0.471	0.984	0.637
ReGVD-structural-local	Majority vote	0.471	0.984	0.637
ReGVD-structural-local	Max-risk	0.460	1.000	0.630

Table 16

Aggregation and selective-prediction diagnostics for exact public checkpoints on CodeXGLUE/Devign. Calibrated rows use post-hoc clean-test thresholds.

Detector	Strategy	Coverage	Precision	Recall	F1
VulBERTa-public	Original fixed	1.000	0.627	0.535	0.577
VulBERTa-public	Max-risk fixed	1.000	0.588	0.645	0.615
VulBERTa-public	Mean probability calibrated	1.000	0.535	0.914	0.675
VulBERTa-public	Mean logit calibrated	1.000	0.535	0.908	0.674
VulBERTa-public	Abstain on disagreement	0.694	0.670	0.449	0.538
LineVul-public	Original fixed	1.000	0.615	0.006	0.013
LineVul-public	Max-risk fixed	1.000	0.517	0.072	0.126
LineVul-public	Mean probability calibrated	1.000	0.460	0.999	0.630
LineVul-public	Abstain on disagreement	0.939	0.667	0.003	0.007

Table 17

Post-hoc LineVul threshold sensitivity on CodeXGLUE clean test scores. These diagnostic thresholds are not used for the primary exact-public results.

Setting	Threshold	Clean recall	Clean F1	Variant F1	Flip
Fixed public threshold	0.500000	0.006	0.013	0.037	0.012
Post-hoc clean-F1 threshold	0.000012	0.998	0.630	0.630	0.002
Post-hoc recall 0.25	0.000043	0.250	0.331	0.410	0.143
Post-hoc recall 0.50	0.000019	0.501	0.491	0.526	0.123
Post-hoc recall 0.80	0.000014	0.799	0.589	0.600	0.070

Summary of RQ4

Variant aggregation is useful as a diagnostic but not yet a complete mitigation. Majority vote changes little, max-risk improves recall at a precision cost, calibrated averaging can improve VulBERTa's post-hoc F1, and abstention exposes disagreement, but deployable use would require validation-set calibration and explicit precision–recall operating policies.

6. Discussion

6.1. Clean Accuracy Is Not Enough

Clean benchmark metrics remain important, but they do not capture prediction stability. A detector can achieve strong clean F1 while failing to preserve predictions under harmless edits. Robustness metrics should therefore be reported alongside standard metrics for vulnerability detection.

6.2. Implications

Prediction instability has direct implications for security automation. If a small behavior-preserving edit converts a true positive into a false negative, then an automated gate may allow vulnerable code to pass. Security-sensitive workflows may prefer conservative aggregation strategies

that increase recall, even when they introduce additional false positives.

Benchmark maintainers should consider releasing transformed variants, reporting transformation validity, and separating robustness test sets from training data. They should also report token lengths and truncation rates because pre-processing artifacts can affect robustness.

Tool builders can use transformation-based tests as part of pre-deployment validation. Prediction instability across code review revisions can also be monitored as an operational signal. Lightweight aggregation over transformed variants is a baseline mitigation idea, but stronger calibrated and selective aggregation methods are needed before deployment.

6.3. Possible Causes of Brittleness

Detector brittleness may arise from token-level shortcut learning, identifier memorization, sensitivity to formatting, truncation artifacts, or weak modeling of data and control dependencies. The relative importance of these causes may vary by detector family and transformation type.

To move beyond marginal flip counts, we fit a diagnostic logistic regression over the exact public CodeXGLUE pairs, predicting whether a transformed variant flips relative to its original. The model includes transformation family,

vulnerability label, original score margin, code length, a token-length proxy, and a token-length truncation proxy. For VulBERTa, identifier renaming has the largest transformation association (odds ratio 223.3 relative to the layout-only reference), followed by safe dead-code carriers (187.7), comment insertion (184.3), dead-branch insertion (168.5), and code normalization (21.2). Larger original score margins reduce flip odds (OR 0.39), consistent with threshold-adjacent examples being more brittle. The token-length truncation proxy is not significant for VulBERTa (OR 1.04, $p = 0.156$), but is significant for LineVul (OR 1.33, $p < 10^{-12}$). These associations are not causal explanations of internal model behavior, but they localize brittleness toward transformation family and calibration margin more strongly than toward length alone.

Our experiments identify where predictions change, but they do not fully explain why each model changes internally. We do not perform token-level attribution, representation probing, or a systematic ablation over transformation strength for the exact public checkpoints. The 512-origin composition and identifier-ablation probe partially addresses interaction effects, but it uses local detector-family baselines and therefore remains supporting evidence rather than a replacement for full exact-checkpoint composition runs.

7. Threats to Validity

7.1. Construct Validity

The main construct validity threat is that a transformation may not preserve semantics in all contexts. We mitigate this risk through conservative transformation rules, syntax checks, deterministic pre-audit, and a completed two-annotator human annotation protocol. The human audit identified no invalid cases but flagged 11 conservative uncertain cases; these were excluded from our analysis to ensure high-fidelity validity claims.

7.2. Internal Validity

Prediction changes may be caused by tokenization, truncation, preprocessing, or thresholding rather than model reasoning. We mitigate this threat by using fixed thresholds, logging token lengths, reporting truncation rates, and applying transformations only at evaluation time.

7.3. External Validity

Function-level datasets may not fully represent whole-project vulnerability detection or deployment in industrial CI/CD systems. We mitigate this threat by using Big-Vul and DiverseVul as EVPs and by limiting claims to function-level vulnerability detection. The strongest detector-specific conclusions are restricted to CodeXGLUE/Devign, where exact public VulBERTa and LineVul checkpoints are executed on the full transformed test set. Big-Vul and DiverseVul support narrower claims about transformation applicability and imbalance-aware metric behavior; they are not evidence for full cross-dataset detector rankings. The

CodeXGLUE/Devign split used here does not provide sufficiently reliable fine-grained CWE labels for a primary per-CWE robustness analysis. Big-Vul and DiverseVul contain CWE metadata, but in this article they are sampled EVPs rather than exact-checkpoint evaluation datasets. Consequently, vulnerability-type-conditioned robustness remains an important extension rather than a central claim of this study.

Detector coverage is also limited. The strongest evidence uses full-test public VulBERTa and LineVul checkpoints, with the official ReGVD graph implementation added on the full transformed CodeXGLUE test set and GraphCodeBERT-Devign added on a meaningful 512-origin exact-checkpoint subset. ReGVD's non-collapsed row depends on validation-threshold calibration; at the upstream fixed 0.5 threshold it collapses to all-safe predictions. The AST-structural-RF row is a working local structural baseline but not a public Devign/ReVeal checkpoint. After discounting the degenerate LineVul fixed-threshold row, the clearest full-test exact-checkpoint robustness signal still comes from VulBERTa, while calibrated ReGVD provides public structural implementation coverage. Modern graph detectors with stronger public checkpoints, UniXcoder vulnerability classifiers, proprietary scanners, and LLM-based zero-shot detectors may behave differently. The contribution is therefore best read as a reusable robustness benchmarking protocol with full-test CodeXGLUE evidence for selected public detectors, not as a comprehensive ranking of vulnerability detectors or a claim about AI-assisted security tools in general.

Some transformations are intentionally conservative but synthetic. For example, `if (0)` dead-code carriers are useful for testing whether a model reacts to unreachable code, yet developers may more often introduce logging, feature-flagged, or preprocessor-guarded code. We therefore interpret dead-code results as stress tests for superficial token reliance rather than frequency estimates of ordinary maintenance edits.

7.4. Conclusion Validity

Observed robustness drops may depend on sample selection or detector configuration. We mitigate this threat by using paired analyses, confidence intervals, multiple detector families, and a reproducible artifact package. For LineVul in particular, the fixed 0.5 function-level threshold may not be the operating point a practitioner would choose after calibration. Alternative thresholds could improve recall and change flip-rate magnitudes, although the reported row remains the exact public checkpoint behavior under the documented runner.

8. Conclusion

This article argues that learning-based vulnerability detectors should be evaluated not only by clean benchmark performance, but also by robustness under semantics-preserving code transformations. Clean F1, PR-AUC, balanced accuracy, and MCC remain important measures of

predictive quality, yet they do not reveal whether a detector preserves its behavior when code is reformatted, renamed, commented, normalized, or rewritten in conservative behavior-preserving ways. For software systems use cases such as code review, continuous integration, and security regression testing, this stability is a first-order software-quality property.

We presented a low-compute empirical protocol for transformation-based verification and validation of vulnerability detection tools. The protocol keeps each transformed variant paired with its original function, applies detector inference without retraining, and reports prediction flip rates, confidence intervals, paired tests, transformed performance, and label-specific fragility. The artifact includes transformed CodeXGLUE, Big-Vul, and DiverseVul samples; exact public VulBERTa and LineVul predictions on the full transformed CodeXGLUE test set; exact public GraphCodeBERT-DeSign predictions on a 512-origin transformed subset; an official ReGVD graph run on the full transformed CodeXGLUE test set; a non-collapsed AST-structural baseline; supporting detector-family baselines; a 512-origin composition and identifier-ablation diagnostic; enhanced aggregation and flip-mechanism analyses; and audit materials for checking transformation validity.

The results show that robustness cannot be inferred from clean benchmark performance alone. On the full transformed CodeXGLUE test set, exact public VulBERTa exhibits a measurable 7.01% prediction flip rate and a clean-to-variant F1 drop from 0.577 to 0.555. Exact public LineVul has a lower flip rate of 1.22%, but this apparent stability is limited by near-zero vulnerable recall. Official ReGVD also illustrates the importance of calibration: its fixed threshold collapses, whereas validation-threshold calibration produces a non-collapsed full-test structural row with a 0.81% flip rate but weak balanced accuracy. Across transformation families, identifier renaming, comment insertion, dead-branch insertion, and safe dead-code carriers are more disruptive than layout-only changes.

The practical implication is that robustness evaluation should become part of the standard reporting and quality-assurance package for vulnerability detectors. Researchers can use transformation tests to expose sensitivity to superficial source features; benchmark designers can report transformed variants alongside clean test sets; and tool builders can use paired robustness tests as pre-deployment checks for learning-based security tools in code-review and CI pipelines. Simple aggregation over transformed variants is only a baseline; calibrated score averaging and selective abstention are more informative diagnostics, but practical deployment still requires validation-set calibration, better model design, training data, and semantic reasoning.

Overall, the strongest empirical claim is deliberately scoped to CodeXGLUE/DeSign, where exact public checkpoints are evaluated on the full transformed test set and the semantic audit validates a randomly sampled subset with strong annotator agreement, no invalid examples, a 95% upper bound of 0.43% on the invalid rate, and conservative

exclusions. Big-Vul and DiverseVul strengthen external validity only in a limited sense: they show that the transformations can be applied to additional vulnerability corpora and demonstrate the need for imbalance-aware metrics. The 512-origin composition probe suggests that composed edits can still produce measurable flips, but the full-test exact-checkpoint claims remain single-transformation claims. This distinction keeps the conclusions useful while avoiding overstatement from sampled, supporting baselines.

Acknowledgement

We acknowledge the support of time and facilities from Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for this study.

Data and Code Availability

The source code, experiment scripts, manuscript sources, and replication materials are available in the GitHub repository <https://github.com/LeoTTTPhat/vd-robustness>.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, DeSign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: *Advances in Neural Information Processing Systems*, 2019, pp. 10197–10207.
- [2] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: *Proceedings of the International Conference on Mining Software Repositories*, 2022.
- [3] H. Hanif, S. Maffei, VulBERTa: Simplified source code pre-training for vulnerability detection, in: *International Joint Conference on Neural Networks*, 2022.
- [4] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. B. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong, M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, S. Liu, CodeXGLUE: A machine learning benchmark dataset for code understanding and generation, *arXiv preprint arXiv:2102.04664* (2021).

- [5] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A c/c++ code vulnerability dataset with code changes and cve summaries, in: Proceedings of the International Conference on Mining Software Repositories, 2020.
- [6] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses, 2023.
- [7] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP, 2020.
- [8] D. Q. Nguyen, K. Rieck, L. Grunske, C. Kästner, Revisiting graph neural networks for vulnerability detection, in: Proceedings of the International Conference on Software Engineering, 2022.
- [9] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, Z. Q. Zhou, Metamorphic testing: A review of challenges and opportunities, ACM Computing Surveys 51 (1) (2018) 1–27. doi:10.1145/3143561.
- [10] N. Yefet, U. Alon, E. Yahav, Adversarial examples for models of code, Proceedings of the ACM on Programming Languages 4 (OOPSLA) (2020) 162:1–162:30. doi:10.1145/3428230.
- [11] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, Z. Jin, Generating adversarial examples for holding robustness of source code processing models, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, 2020, pp. 1169–1176. doi:10.1609/aaai.v34i01.5469.
- [12] P. Bielik, M. Vechev, Adversarial robustness for code, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 896–907.
- [13] M. R. I. Rabin, N. D. Q. Bui, K. Wang, Y. Yu, L. Jiang, M. A. Alipour, On the generalizability of neural program models with respect to semantic-preserving program transformations, Information and Software Technology 135 (2021) 106552. doi:10.1016/j.infsof.2021.106552.
- [14] J. Henkel, G. Ramakrishnan, Z. Wang, A. Albarghothi, S. Jha, T. Reps, Semantic robustness of models of source code, in: Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering, 2022.
- [15] Z. Yang, J. Shi, J. He, D. Lo, Natural attack for pre-trained models of code, in: Proceedings of the International Conference on Software Engineering, 2022, pp. 1482–1493. doi:10.1145/3510003.3510146.
- [16] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, Z. Jin, Towards robustness of deep program processing models: Detection, estimation, and enhancement, ACM Transactions on Software Engineering and Methodology (2022).
- [17] S. Wang, Z. Li, H. Qian, C. Yang, Z. Wang, M. Shang, V. Kumar, S. Tan, B. Ray, P. Bhatia, R. Nallapati, M. K. Ramanathan, D. Roth, B. Xiang, ReCode: Robustness evaluation of code generation models, in: Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2023.
- [18] Y. Li, S. Qi, C. Gao, Y. Peng, D. Lo, D. Y.-A. Lo, M. R. Lyu, Z. Xu, Understanding the robustness of transformer-based code intelligence via code transformation: Challenges and opportunities, IEEE Transactions on Software Engineering 51 (2) (2025) 521–547.
- [19] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet?, IEEE Transactions on Software Engineering (2022).
- [20] L. Sun, A. Oprea, E. Wong, Semantics-preserving evasion of llm vulnerability detectors, arXiv preprint arXiv:2602.00305 (2026). arXiv:2602.00305.
- [21] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, M. Zhou, GraphCodeBERT: Pre-training code representations with data flow, in: International Conference on Learning Representations, 2021.

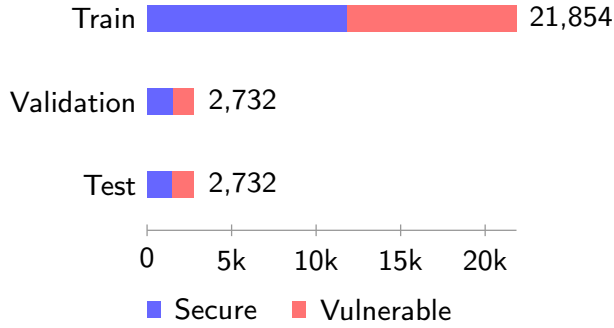


Figure 2: Class composition of the normalized CodeXGLUE/Devign splits.

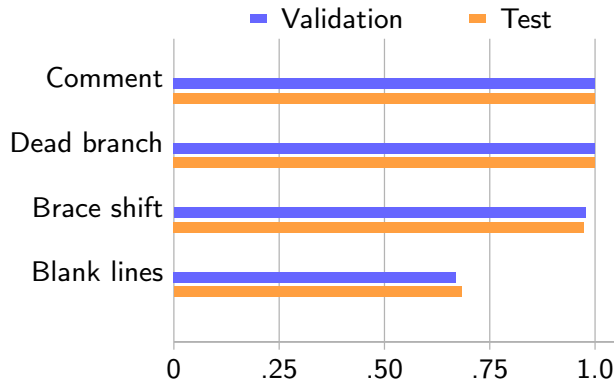


Figure 3: Transformation applicability on the CodeXGLUE/Devign validation and test splits.

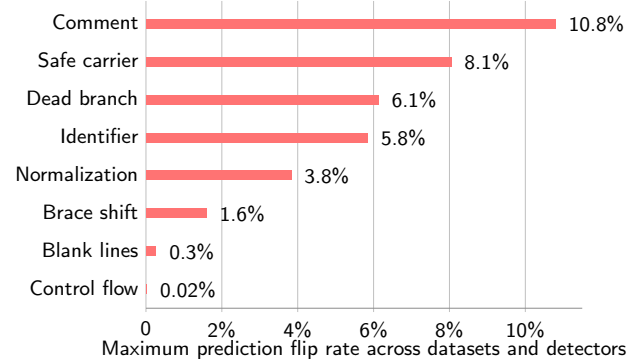


Figure 5: Maximum observed prediction flip rate by transformation across datasets and executed detector-family baselines.

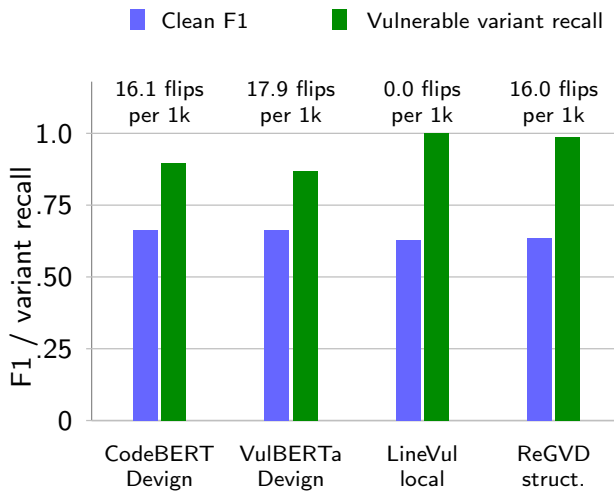


Figure 4: CodeXGLUE clean F1, vulnerable variant recall, and prediction flip rate for the executed detector-family baselines.

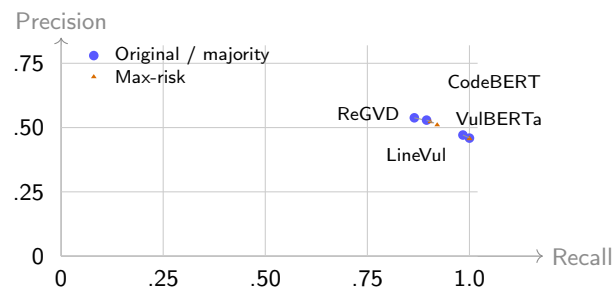


Figure 6: Precision–recall tradeoff of max-risk aggregation on CodeXGLUE/Devign.